

Student Meal Planner

Frontend Implementation

An Implementation Document for the Grubs4Scrubs Project

Date: April 2026

Author: Yao Cheng Zhou

What This Document Covers

This is a walkthrough of the Grubs4Scrubs frontend as it stands right now. Each page gets a section explaining what it does, how it's built, and what changed during development. The biggest change was stripping out Tailwind CSS entirely and rebuilding everything in custom CSS. I'll cover that migration and what the pages look like after it.

[NOTE TO SELF: Add screenshots of each page here when the app is running. A before/after comparison of the Dashboard would be especially good.]

Tech Stack

React 19 with Vite as the build tool. No component libraries. No state management beyond `useState` and `useContext`. Routing with `react-router`. Icons from `lucide-react`. API calls through `Axios` (base URL: `http://localhost:5075/api`). Custom CSS with the `ComponentName-element-subelement` naming convention. No Tailwind.

Home Page

The landing page. Five sections stacked vertically: hero banner, feature cards, featured recipes, how-it-works steps, and a sign-up CTA. Each section is its own function inside `HomePage.jsx`.

The featured recipes section is interactive. Clicking a recipe card navigates to `/recipes/:id` using `React Router's useNavigate`. That wiring was added later in development after the Recipe View page was built.

The page went through a couple of rough patches. At one point an accidental import (`import RecipeView from "../RecipeViewPage.jsx"`) caused a 500 error on load. Another time the file got truncated and was missing closing tags on the `HomeCreateAccountTab` function. Both were caught and fixed.

Dashboard Page

This page had the biggest transformation. It started as pure Tailwind. Dense utility strings everywhere. Something like `className="bg-white/[0.04] border border-white/[0.06] p-6 rounded-card"` on every div. Reading the JSX meant mentally parsing twelve classes just to understand what a container looked like.

After the migration to custom CSS, that same div became `className="Dashboard-stat-card"`. The styles moved to `DashboardPage.css` where they're easy to find and modify. The JSX went from cluttered to readable.

The page has five sections: a greeting header with the current date, metric cards (budget spent, saved recipes, planned meals, shopping items), today's meal plan split into breakfast/lunch/dinner columns, featured recipe suggestions, and a mini shopping list. The shopping list uses `useState` to track which items are checked off.

Recipes Page

A grid of recipe cards with category filter buttons at the top. Clicking "Breakfast", "Lunch", "Dinner", or "Snack" filters the visible cards. An "All" button resets the filter.

Each card shows an image, title, tags, prep time, and estimated budget. Cards are clickable and navigate to the Recipe View page. There's also a dashed-border "add recipe" card for creating new recipes (not yet wired to a creation form).

One bug that came up during development: `recipe.name?.toLowerCase()` was throwing a `TypeError` because some recipe objects used `title` instead of `name`. Optional chaining fixed the crash, but the real fix was making sure the data model was consistent.

Recipe View Page

This is the detail page that opens when you click a recipe card. It uses `useParams()` to grab the recipe ID from the URL and fetches the recipe from the backend API.

The page has a hero section at the top with a full-width image (or an emoji fallback if there's no image), a gradient overlay, and a meta bar showing prep time, cook time, and servings. Below that: a checkable ingredient list (`useState` toggles each checkbox), a nutrition grid, step-by-step instructions with numbered circles, and a tip card with an orange glow effect.

The ingredient checklist was something I wanted specifically because I've used recipe sites where you lose track of which ingredients you've already grabbed. Being able to tap each one off the list is a small thing but it makes a difference.

Two action buttons sit in the hero section: an edit button (pencil icon) and a delete button (trash icon). The delete button sends a `DELETE` request to the API and redirects back to the recipes grid. The edit button opens a modal pre-filled with all the recipe's current data. Ingredients and instructions get parsed from their JSON format back into readable text (one item per line) for editing, then converted back to JSON on save. The modal sends a `PUT` request to update the recipe and refreshes the displayed data.

Meal Planner Page

Weekly meal planning. Day tabs across the top (Monday through Sunday), a three-column grid showing Breakfast, Lunch, and Dinner slots for the selected day, and a week overview at the bottom with total meals and estimated cost.

The day tabs use `useState` to track which day is selected. Originally the week overview used `.reduce()` to calculate totals, but I replaced that with a simple nested for loop because I couldn't fully follow what `reduce` was doing. The for loop is more verbose but I understand every line of it.

```
let totalCost = 0
let totalMeals = 0
for (const dayMeals of Object.values(sampleMeals)) {
  for (const meal of Object.values(dayMeals)) {
    if (meal) {
      totalCost = totalCost + meal.cost
      totalMeals = totalMeals + 1
    }
  }
}
```

Empty slots show a dashed border with a plus icon. Filled slots show the meal image, name, and cost. The CSS for this page was written from scratch (no Tailwind conversion needed since the page didn't exist before the migration).

Shopping List Page

The shopping list page is fully connected to the backend API. Items are fetched on load, and every action (add, edit, delete, check off) hits the API immediately.

Adding an item opens input fields at the top of the list for name, quantity, and price. Submitting sends a POST request and the new item appears in the list. Each item has two action buttons: a pencil icon for editing and a trash icon for deleting.

The edit feature uses an inline editing pattern. Clicking the pencil sets an `editingId` state variable to that item's ID, which swaps the text display for input fields pre-filled with the current values. A check button saves the edit (PUT request) and an X button cancels without saving. Only one item can be edited at a time. The editing row gets a blue-tinted background so it's visually distinct.

Checking an item off toggles the `isChecked` boolean and sends a PUT request to persist the state. The item gets a strikethrough style when checked.

Login Page and Sign-Up Page

Both pages exist as frontend components with forms built and styled, but they're not connected to the backend auth API yet. That wiring is planned for the next submission.

LoginPage has email and password fields, a submit button, and a "Sign in with Google" button (not functional yet). SignUpPage has username, email, password, and confirm password fields. Both pages follow the same dark theme and glass-card styling as the rest of the app.

The backend endpoints they'll connect to (`/api/auth/login` and `/api/auth/register`) are fully functional. The gap is just the Axios calls and token storage on the frontend side.

The Tailwind Removal

Worth documenting because it touched almost every file. The process was:

1. Identify which pages used Tailwind (Dashboard was the worst offender)
2. Create a matching .css file with custom classes using the `ComponentName-element-subelement` convention
3. Replace every `className="utility-string"` in the JSX with the new semantic class name
4. Remove the Tailwind imports (`@import "tailwindcss/theme.css"` and the `@theme` block from the CSS)
5. Test that everything still looks the same

After the migration, the project has zero Tailwind dependencies. Every page uses the same color palette, the same naming convention, and the same CSS patterns (glass cards, hover effects, accent colors). The consistency wasn't there before.

What's Left to Build

Recipe CRUD and shopping list CRUD are fully wired to the backend. The remaining work is:

- Connect LoginPage and SignUpPage to the auth API endpoints (store JWT token, send with requests)
- Make the meal planner persist plans to the database (POST/PUT to the API)
- Tie shopping list items to user accounts (currently shows all items regardless of who's logged in)
- Add recipe ownership enforcement (only the creator can edit/delete their recipes)

- Add error handling and loading states for all API calls
- Fix the HomePage footer positioning (known CSS issue with absolute positioning)